Problem Statement Handout — `stor`

*Part 2 of the briefing. This is the participant-facing reference for the program you build and attack. The published **functionality test suite is the authoritative definition** of correct behavior where the English below is ambiguous.*

---

1. The program

`stor` is a command-line **encrypted file storage system**, written in **C**. Users register with a username and a **secret key**; they create files and read/write data to them. All state is persisted to a single encrypted database file, **`enc.db`**, in the working directory. The key protects the data — without it, nobody should be able to read or modify a user's files. **`stor` must work without Internet access.**

```
stor -u alice -k secret123 register            # create an account
stor -u alice -f notes create                    # create a file (no key needed)
stor -u alice -k secret123 -f notes write "Hello"   # write to it
stor -u alice -k secret123 -f notes read            # → "Hello"
```

Required behaviors

- `register` — register a user with a username and key.
- `create` — create a file owned by a user (**does not** require the key).
- `write` — write content to a file (**requires** the key).
- `read` — read a file's content (**requires** the key).
- All state lives in `enc.db`.
- The program **must** contain a `win()` function that prints `Arbitrary access achieved!`. It does nothing during Build-It, but is the target of the Break-It control-flow category.

2. Command-line specification

```
stor -u <username> (-k <secretkey>) [read|write|register|create] \
     (-f filename) (-i inputfile) (-o outputfile) <text>
```

Argument	Meaning
`-u`	Username used to authenticate / register.
`-k`	Secret key used to authenticate / register.
`register`	Register a user (needs `-u` and `-k`).
`create`	Create a file (needs `-u` and `-f`; **no** `-k`).
`write`	Write content to a file (needs the key).
`read`	Read a file's content (needs the key).
`-f`	Filename in the encrypted store.
`-i`	If present, read input from this existing file; otherwise input is the inline `<text>`.
`-o`	If present, write output to this new file; otherwise output goes to stdout.
`<text>`	Inline input when `-i` is absent (used with `write`).

- `register`, `read`, `write` require a key; `create` does not.
- A named argument may be supplied multiple times; the **last occurrence wins** (`-k key1 -k key2` ⇒ key is `key2`).

3. Return values & errors

On any error, or an incomplete/contradictory argument combination, `stor` must print **`invalid`** to stdout and exit with code **255**.

4. Security model (this is the real challenge)

Without a user's secret, an attacker must **not** be able to:

- **Confidentiality** — learn facts about a user's files by inspecting `enc.db` directly.
- **Integrity** — modify another user's data.
- **Forgery detection** — any modification to `enc.db` made other than by correct use of `stor` must be

detected when the correct secret is later supplied.

The C program must be written with **memory safety** in mind (no leaks, dangling pointers, use-after-free). The one exception: on a very large write with no available memory/disk, the program may exit **gracefully** — but it must **not crash**.

5. Break-It categories & points

For each target team you submit machine-checkable test cases in one of these categories:

Category (`type`)	Points	Demonstrates
Correctness	25	Output / return code deviates from the spec.
Crash	50	Segfault, or a command that takes longer than 60 s (DoS).
Integrity / **Confidentiality** / **Security**	100	Breaking the security model — modify a victim's file, recover secret content, or hijack execution to `win()`. Submitted as distinct `type`s but **scored the same**.

There are no penalties for incorrect attempts, but each counts as a submission attempt. Each unique bug is worth its severity above. In the **final standings**, a bug found by several teams counts **once** and its points are **split** among the finders. (The live scoreboard is provisional and shows each break in full.)

Break package

For each qualifying target, organizers publish the team's source plus a standard `enc.db` generated as below (the secret key is **not** given to you). The grader **recompiles the target's source from their repo** before testing — your exploit must work against the freshly built binary.

```
stor -u user1 -k <secret> register
stor -u user1 -f file1 create
stor -u user1 -k <secret> -f file1 write "<integrity target content>"
stor -u user2 -k <secret> register
stor -u user2 -f file2 create
stor -u user2 -k <secret> -f file2 write "<confidentiality target content>"
```

- **Integrity:** change `file1`'s content so the victim's `stor` reads your modified content as valid.
- **Confidentiality:** recover `file2`'s secret content.

Test-case format

```
break/<your-test-name>/
├── test.json        # required: the machine-checkable test
└── description.txt   # required: human explanation of the bug + how the test trigg
```

```
{
  "target_team": 12,
  "type": "security",
  "commands": [
    {"program": "stor", "args": ["-u","alice","-k","secret123","register"]},
    {"program": "stor", "args": ["-u","alice","-f","exploit","create"]},
    {"program": "stor", "args": ["-u","alice","-k","secret123","-f","exploit","writ
     "stdin": "AAAA...BBBBCCCC"}
  ]
}
```

- `target_team` — the **numeric team ID** shown on the dashboard.
- `type` — one of `correctness`, `crash`, `integrity`, `confidentiality`, `security`.
- `commands` — the sequence of `stor` invocations; each may include an optional `stdin`, and (for **confidentiality**) an `output` field with the secret value you claim to have recovered — the grader compares it to the real value.

```
{ "target_team": 12, "type": "confidentiality",
  "commands": [
    {"program": "stor", "args": ["-u","user2","-f","file2","read"],
     "output": "the_secret_you_recovered"}
  ]
}
```

Break submission rules

- **Both `test.json` and `description.txt` are required** — a break missing the description is rejected.
- A break is created/updated only when a commit **adds or modifies `break/<name>/test.json`**. Commits touching only `description.txt` won't register.

- Test input files must not exceed **600 MB**; each command is killed after **60 s** (for the crash category, a timeout counts as a successful break).

Rules: don't attack the contest infrastructure itself; each team does its own work; share how you broke another team with them only after the relevant deadline.